

HACKATHON HANDBOOK

Atomic Multi-Agent Service Router

Agentic Payments using x402 on Algorand

A complete, beginner-friendly reference book for teams building a pay-per-use, multi-agent AI system with atomic, on-chain payments.

How to use this book

This book is written so that **two teammates with only basic programming knowledge** can read it once and immediately start building. You do not need to know blockchain, Algorand, or x402 before you start — every term is explained the first time it appears.

Read Chapters 1–9 together before you write any code — they build the mental model. From Chapter 10 onward, you can split up: one person can build the router and agents while the other builds the payment and blockchain layer, using this book as the shared reference.

Keep this document open on a second screen during the entire hackathon.

Table of Contents

1. Understanding the Problem Statement
2. What Does "Atomic" Mean?
3. Understanding Multi-Agent AI
4. Understanding the Router
5. Understanding x402
6. The Complete x402 Payment Flow
7. Understanding Algorand
8. Understanding the Facilitator
9. Complete System Architecture
10. API Design
11. Business Model
12. Real Paying Users
13. Unique Innovation Ideas
14. Technology Stack
15. Folder Structure
16. Database Design
17. Error Handling
18. Security
19. Demo Flow (5 Minutes)
20. Presentation Slides (8 Slides)
21. Judges' Questions (50+) with Model Answers
22. Comparison with Existing Tools
23. Future Scope
24. Complete Development Roadmap

Chapter 1 — Understanding the Problem Statement

The one-sentence version

You are building a **traffic controller for AI agents that also handles money** — a single "router" that takes one request from a user, breaks it into smaller jobs, sends each job to the right specialist AI agent, **pays each agent automatically for its work using real (testnet) money on Algorand**, and returns one combined answer — and if anything goes wrong partway through, **no money is lost and nothing is left half-done**.

That's it. Every chapter in this book is really just zooming into one word of that sentence.

What is the Atomic Multi-Agent Service Router?

Think of it as a **smart receptionist + accountant**, combined into one piece of software.

- As a **receptionist**, it listens to what the user wants, figures out which specialists are needed, and coordinates them.
- As an **accountant**, it makes sure every specialist gets paid exactly for the work it did, the payment is provable, and if a specialist fails to deliver, the money is not wasted.

The "specialists" here are not humans — they are **AI agents**, and instead of paying them with salaries, we pay them **per API call**, automatically, using a payment protocol called **x402**, settled on a blockchain called **Algorand**.

Why does this problem exist?

Today, when software wants to use a paid AI service (like a translation API or a weather API), a **human** usually has to:

1. Sign up for an account
2. Enter a credit card
3. Subscribe to a monthly plan
4. Get an API key
5. Hard-code that key into the app

This works fine when a human is setting things up once. But it breaks down when **AI agents themselves** need to hire other AI agents automatically, on the fly, without a human clicking "subscribe" every time.

Imagine your AI travel assistant realizes, in the middle of a conversation, that it needs a live currency conversion from a paid API it has never used before. It cannot stop and ask you to enter a credit card. It needs to **pay for that single API call itself, instantly, and prove it paid** — all in code, in milliseconds.

What real-world issue is it solving?

It solves the "**machine-to-machine commerce**" **problem**: how do two pieces of software that have never met before transact safely, without a human in the loop, and without a long-term contract?

Today's internet has no native way for one program to pay another program for a single unit of work. x402 fixes this by reviving an old, unused HTTP status code — **402 Payment Required** — and giving it real meaning.

Why is a normal AI assistant not enough?

A single AI assistant (like a general chatbot) is a **generalist**. Generalists are good at many things but excellent at almost nothing. If you ask a single AI to book a flight, find a hotel, check the weather, and process a payment, it will try to do all of these using one broad set of skills, and it will:

- Make more mistakes on specialized tasks (like reading live flight data correctly)
- Be unable to independently verify its own work
- Be a single point of failure — if it hallucinates on one part, the entire answer breaks
- Be unable to be individually priced, scaled, or replaced

Why are multiple AI agents better than one AI?

A **multi-agent system** splits the big job into small jobs and gives each small job to a **specialist agent** that is really good at just that one thing. This mirrors how real organizations work — a hospital doesn't have one person who is the receptionist, doctor, pharmacist, and billing clerk all at once.

Benefits of specialization:

One Big AI	Multiple Specialist Agents
Tries to do everything	Each agent does one thing well
Hard to improve one skill without affecting others	You can upgrade one agent without touching the rest
Cannot be individually priced	Each agent can charge its own price per call
One failure breaks everything	One agent's failure can be isolated and retried or replaced
Hard to scale	Each agent can scale independently (more servers for the popular one)

Story format: five real-life examples

1. Travel booking. Ananya asks her AI assistant: *"Plan me a 3-day trip to Goa under ₹20,000."* Behind the scenes, a **Router** wakes up a **Flight Agent**, a **Hotel Agent**, a **Weather Agent**, and a **Restaurant Agent**. Each one does its specialized search, each one is paid a small fee the instant it responds, and the Router stitches their answers into one itinerary. If the Hotel Agent fails to respond, the Router does not pay it, and it tries a backup Hotel Agent instead — Ananya never even notices.

2. Online shopping. A shopping AI wants to compare prices across five stores. Each store exposes a **paid** price-check API (to stop scrapers from hammering it for free). The Router pays each store 10 paise per lookup via x402, gets five real-time prices back, and only actually purchases from the cheapest one. Money only moves for lookups that were actually used.

3. Hospital assistant. A patient's AI assistant asks a **Symptom-Checker Agent**, a **Pharmacy-Stock Agent**, and an **Insurance-Verification Agent** three separate questions at once. Because this involves health data and money, every step must be **atomic**: if the Insurance-Verification Agent fails, the whole booking must roll back — you don't want a patient billed for a consultation that was never actually insured.

4. Insurance claim processing. A claims-processing Router calls a **Document-OCR Agent** to read a submitted bill, a **Fraud-Detection Agent** to check for anomalies, and a **Policy-Rules Agent** to check coverage. Each agent is paid per claim it processes — the insurance company doesn't pay a flat monthly SaaS fee for a tool it uses only occasionally.

5. Business automation. A small business's AI Router handles incoming customer emails by routing them to a **Sentiment Agent**, a **Language-Translation Agent**, and a **Reply-Drafting Agent** — paying each a fraction of a rupee per email, only when that agent is actually used. No subscriptions, no wasted spend on idle tools.

In every example, the same shape repeats: **one request** → **many specialist agents** → **automatic per-call payment** → **one combined, trustworthy answer**. That shape is exactly what you are building in this hackathon.

Chapter 2 — What Does "Atomic" Mean?

Explain it like I'm 10

Imagine you order a pizza online.

Step 1: You pay ₹300 for the pizza. Payment succeeds.

Step 2: The restaurant looks at the order and says, "Sorry, we're closed, we can't make this pizza."

Question: Do you still lose your ₹300?

Obviously not — that would be unfair and no one would ever use that app again. The correct behavior is: since Step 2 failed, Step 1 must be **undone**. Your ₹300 must come back to you. Either **both steps fully happen** (you pay AND you get pizza), or **neither step happens** (you don't pay AND you don't get pizza). There is no in-between world where you paid but got nothing.

This all-or-nothing property is called **atomicity**, and an operation that has this property is called **atomic**.

The formal definition

*An operation is **atomic** if it is indivisible: it either completes **entirely**, or it has **no effect at all**, as if it never started.*

The word comes from the Greek *atomos*, meaning "cannot be cut." Just like you can't have "half an atom" and still call it the same substance, you can't have "half a payment" and call it a completed transaction.

Applying this to our project

In our Atomic Multi-Agent Service Router, a single user request might trigger **three paid agent calls**: Flight Agent, Hotel Agent, Weather Agent. Imagine:

```
Step 1: Pay Flight Agent -> SUCCESS
Step 2: Pay Hotel Agent -> SUCCESS
Step 3: Pay Weather Agent -> FAILS (agent is offline)
```

If we are not atomic, the user has now paid for a flight and a hotel but received no weather data, and worse — the Router might present an incomplete itinerary as if it were complete. This is exactly the "half a pizza" problem.

Being atomic means the Router must guarantee one of two outcomes:

- **Outcome A (Success):** All required agents respond and are paid. The final combined result is returned to the user.
- **Outcome B (Rollback):** At least one required agent fails. Any payments already made for *that* logical transaction are refunded or the workflow is marked as failed *before* being shown to the user as complete, and the user is not charged for the parts that never delivered value.

What is "rollback"?

Rollback means undoing everything that happened so far in a multi-step process, to bring the system back to the state it was in before the process started — as if it had never begun.

In a database, rollback usually means reversing database writes. In our payment context, rollback can mean:

- Not settling a payment that was only *authorized* but not yet finalized
- Issuing a refund transaction on-chain if a payment already settled but the corresponding service was never delivered
- Marking the entire multi-agent task as **FAILED** and not billing the user's final invoice for the failed portion

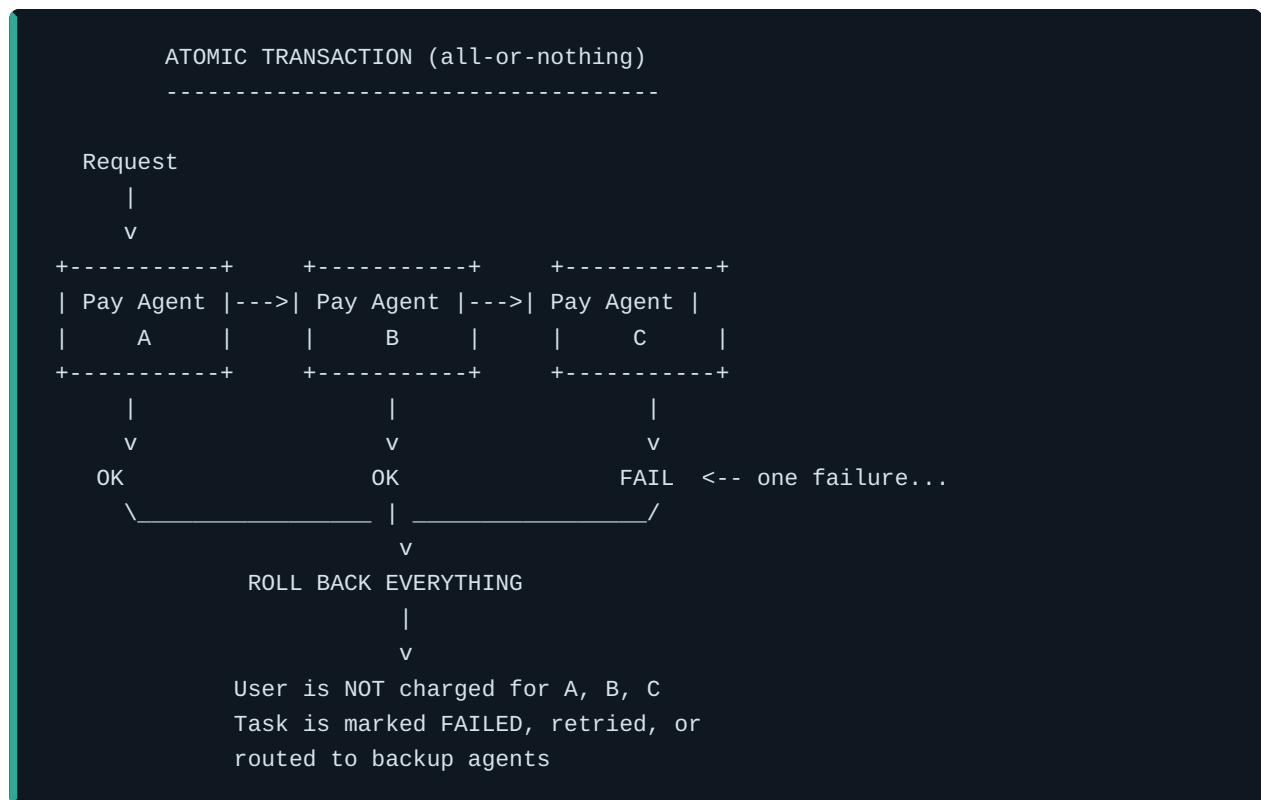
Why atomicity matters so much for money

Money has a special property that other data doesn't: **once it moves, someone feels it**. If a text field is wrong, you fix it and no one is hurt. If a payment is wrong, someone is short ₹300 and rightfully angry.

Three concrete reasons atomicity is non-negotiable here:

1. **Trust.** Machine-to-machine payments only work at scale if every party can trust that "partial failure" never means "partial loss." One bad experience destroys trust in the whole system.
2. **No customer support in the loop.** A human customer service agent can manually refund a mistaken payment. In an autonomous multi-agent economy, there is no human watching every transaction — the system itself must guarantee correctness.
3. **Composability.** If Agent A calls Agent B which calls Agent C, and each hop is atomic, you can safely chain any number of agents together and still trust the whole chain. If even one hop isn't atomic, the risk of loss compounds with every additional agent in the chain.

A simple ASCII picture



Keep this diagram in your head for the rest of the book — it is the core promise your Router must keep.

Chapter 3 — Understanding Multi-Agent AI

What is an AI agent?

An **AI agent** is a program that can **perceive** an input, **decide** what to do about it, and **act** — usually by calling tools, APIs, or other agents — to accomplish a goal, often with some autonomy (it can make small decisions without being told exactly what to do at every step).

The key difference from a plain script is **decision-making**: a script always does the same steps in the same order; an agent can look at the situation and choose *which* steps to take.

Chatbot vs. AI Assistant vs. AI Agent vs. Multi-Agent System

Term	What it does	Example
Chatbot	Answers questions with text. No real-world actions.	A FAQ bot that answers "What are your opening hours?"
AI Assistant	Chatbot + can call a few tools/APIs on your behalf, usually within one app.	Booking a single calendar event when you ask it to.
AI Agent	Has a goal, can plan multiple steps, call tools/APIs, and adapt if something fails, with some independence.	An agent that searches flights, compares five options, and books the cheapest one without being told each step.
Multi-Agent System	Multiple specialized agents, each an expert in a narrow area, coordinated by something (a router/orchestrator) to solve a bigger problem together.	A travel-planning system with separate Flight, Hotel, Weather, and Payment agents working together.

Example: the Travel Planner multi-agent system



Each specialist agent only needs to be good at **one narrow job**. The Hotel Agent doesn't need to know anything about flights, and the Flight Agent doesn't need to know anything about weather. This narrow focus is exactly what makes each agent more accurate and more trustworthy than a single do-everything AI.

Why does each specialized AI perform better than one giant AI?

1. **Narrower context = fewer mistakes.** A specialist agent's prompt and tools are focused only on its domain, so it is less likely to get confused or hallucinate about unrelated things.
2. **Independent upgrades.** You can swap the Flight Agent for a better one without touching the Hotel Agent's code at all.
3. **Independent pricing.** Each agent can charge a fair price for its own specific work — a heavy OCR agent might cost more per call than a simple weather lookup.
4. **Parallelism.** Independent agents can all work **at the same time** instead of one AI doing everything step-by-step, which makes the whole system faster.
5. **Fault isolation.** If one agent is buggy or down, only that one piece of the system is affected — the Router can retry, use a backup agent, or gracefully degrade instead of the entire system failing.

This is the same reasoning that led real companies to move from giant "monolith" applications to smaller "microservices" — except here we're applying it to AI agents instead of backend services, and adding a payment layer on top so each microservice-like agent can charge for its own usage.

Chapter 4 — Understanding the Router

What is a router?

The **Router** is the brain of your entire project. It is the single entry point that a user (or another program) talks to. It does **not** do the specialized work itself — it decides **who** should do the work, sends them the work, watches what comes back, and assembles the final answer.

Think of the Router as the **project manager** in a company: it doesn't personally write the code, design the poster, and count the money — it delegates each job to the right specialist and makes sure everything comes together correctly and on budget.

Why do we need a router?

Without a router, the user (or the calling application) would need to know about every single agent, call each one manually, handle each payment manually, and merge the answers manually. That's a lot of complexity to put on the user. The Router hides all of that complexity behind **one simple request**.

Responsibilities of the router

1. **Receive request** — accepts the user's goal (e.g. "plan my trip") through a single API endpoint.
2. **Choose agents** — decides which specialist agents are needed for this particular request, and in what order (some agents might depend on the output of others).
3. **Send requests** — calls each chosen agent's API.
4. **Handle payment challenges** — when an agent responds with `402 Payment Required`, the Router (via its Payment module) signs and sends payment, then retries the call.
5. **Collect responses** — gathers each agent's output as it comes back, potentially running several agents in parallel.
6. **Handle failures** — if an agent fails or times out, decides whether to retry, use a backup agent, or fail the whole task atomically.
7. **Manage payments** — keeps a running ledger of what was paid to whom, and ensures no agent is paid for work it did not successfully deliver.
8. **Generate final response** — merges all the specialist outputs into one clean, useful answer for the user, along with a receipt of what was paid.

Router flow diagram

```
User
|
| POST /task { "goal": "Plan a 3-day Goa trip under 20000" }
v
+-----+
|                                     |
|                               ROUTER |
|                                     |
| 1. Parse goal -> decide which agents are needed |
| 2. For each needed agent: |
|   call agent ---> receive 402 Payment Required |
|   pay agent (via x402 + Algorand) ---> retry call |
|   receive agent's real response |
| 3. If any REQUIRED agent fails -> mark task FAILED, |
|   no charge for undelivered parts (atomic rollback) |
| 4. Merge all successful responses into one answer |
| 5. Attach payment receipt(s) |
+-----+
|
|
v
Final Answer + Receipt returned to User
```

A tiny router sketch (Node.js / Hono, pseudocode)

```
// router.js (simplified for teaching purposes)
import { Hono } from "hono";
const app = new Hono();

app.post("/task", async (c) => {
  const { goal } = await c.req.json();

  // 1. Decide which agents this goal needs
  const plan = decideAgents(goal); // e.g. ["flight", "hotel", "weather"]

  const results = {};
  const receipts = [];

  for (const agentName of plan) {
    try {
      // callAgentWithPayment handles the full x402 flow internally
      const { data, receipt } = await callAgentWithPayment(agentName, goal);
      results[agentName] = data;
      receipts.push(receipt);
    } catch (err) {
      // one required agent failed -> roll back the whole task
      return c.json({ status: "FAILED", reason: err.message }, 502);
    }
  }

  return c.json({ status: "SUCCESS", results, receipts });
});

export default app;
```

Don't worry about `callAgentWithPayment` yet — that function is exactly what Chapters 5 and 6 explain in detail.

Chapter 5 — Understanding x402

What is x402?

x402 is an open payment protocol that gives real meaning to a very old, mostly-unused HTTP status code: **402 Payment Required**. When a server wants to charge for an API call, instead of demanding a subscription or an API key up front, it simply replies to a request with **HTTP 402** and a small JSON body describing **how much** to pay, **in what currency**, and **to what address**. The client then pays (usually on a blockchain, instantly), retries the exact same request with proof of payment attached, and the server serves the request.

In short: **x402 turns "pay to access this API" into a standard, machine-readable HTTP conversation** — no accounts, no sign-up forms, no human required.

Why was x402 created?

The HTTP protocol has had a status code called **402 Payment Required** reserved **since 1997**, but it was never actually implemented — there was no standard way to say "how much" and "how to pay" in machine-readable form, and no fast, cheap, global payment rail that a computer could use instantly. x402 was created once two things became available: (1) stablecoins like USDC that let you send exact small amounts of money instantly and cheaply, and (2) a real need — AI agents that must pay each other without any human clicking "buy."

Why not Stripe?

Stripe is excellent for **human-initiated, account-based** payments — a person enters a card once, and Stripe charges it monthly or per checkout. But Stripe payments involve:

- Creating a merchant account and going through approval
- Storing and reusing card details (a security responsibility)
- Settlement delays (money doesn't move for days)
- Fees that make **very small, per-call payments** (like ₹0.50) impractical

For a machine that wants to pay a tiny amount for a single API call, right now, without ever registering an account, Stripe's flow is far too heavy.

Why not PayPal?

Same fundamental issue as Stripe: PayPal is designed around **human accounts, logins, and checkout flows**. It has no native way for one anonymous piece of software to programmatically pay another piece of software in one HTTP round-trip, with the payment itself acting as proof of authorization.

Why normal APIs require API keys

Most paid APIs today use **API keys**: you sign up once, get a secret key, and put that key in every request's header so the server can identify you and bill your account later (usually monthly). This requires:

- A human sign-up step
- A billing relationship set up **in advance**
- Trust that you won't leak or misuse the key
- The provider maintaining user accounts and monthly invoices

None of this fits a world where an AI agent wants to try a **brand-new** API it has never used before, for a **single call**, with **no prior relationship**.

How x402 enables automatic machine-to-machine payments

x402 replaces "give me your API key" with "pay me for this exact request, right now, and I'll prove you don't need an account." The payment itself — cryptographically signed and settled on a blockchain — **is** the authorization. There is no separate login system to hack, no key to leak, and no invoice to dispute weeks later.

Key ideas x402 introduces

- **Machine-to-machine payments** — software pays software directly, no human approval step per transaction.
- **Pay-per-call APIs** — you pay only for the exact request you make, not a flat monthly fee.
- **No subscriptions** — there's nothing to "cancel"; every call is a fresh, self-contained transaction.
- **No manual checkout** — no cart, no "buy now" button, no redirect to a payment page. It all happens inside the HTTP request/response cycle.

Chapter 6 — The Complete x402 Payment Flow

The flow, step-by-step



Explaining each step

Step 1 — Initial request. The Router calls an agent's API completely normally, as if it were free — no special headers, no payment info attached yet.

Step 2 — Challenge (402 Payment Required). The agent's server doesn't do the paid work yet. It replies immediately with HTTP status **402** and a small JSON payload that says exactly: how much to pay, in which token (e.g. USDC), which blockchain network, and which wallet address to pay. This is the "challenge."

Step 3 — Client signs payment. The Router's **wallet** (a piece of code holding a private key) creates a payment authorization matching the challenge, and **cryptographically signs it**. Signing proves "the owner of this wallet approved this exact payment" without needing a password or login — it's math, not trust.

Step 4 — Retry request. The Router sends the **exact same request again**, this time including the signed payment proof in a header (commonly `X-PAYMENT`). This is still just one more HTTP request — nothing has touched the blockchain yet.

Step 5 — Facilitator verifies payment. The server doesn't try to understand blockchain signatures itself. It hands the signed payment to a helper service called the **Facilitator** (fully explained in Chapter 8), which checks that the signature is valid and that the amount matches what was requested.

Step 6 — Settlement. Once verified, the Facilitator actually **submits the transaction to the Algorand blockchain**, moving the payment from the client's wallet to the server's wallet. This step produces a **transaction ID** — permanent, public proof the payment happened.

Step 7 — Server processes request, receipt returned. Only now does the server do the actual paid work (e.g. run the flight search) and return the real data, along with a receipt (often the transaction ID) in a response header like `X-PAYMENT-RESPONSE`.

Why payment happens *before* the API runs

Because the server has no other reliable way to know the client is serious — there's no account, no invoice, no trust relationship. By requiring **proof of a real, settled (or settling) payment before doing the paid work**, the server protects itself from ever doing work for free. This is the same logic as "pay before the waiter brings your food" at a counter-service restaurant, instead of "pay after eating" — it removes the risk of walking away without paying.

How fraud is prevented

- **Cryptographic signatures** mean only the true owner of a wallet's private key could have authorized a payment — it can't be forged.
- **Each payment challenge includes a unique nonce** (a random one-time number), so a captured payment proof cannot be **replayed** to pay twice for the price of one (this is called a "replay attack" — covered again in Chapter 18).
- **The Facilitator independently verifies** the payment before the server trusts it — the server never has to blindly believe the client's claim of "I paid."
- **Settlement on a public blockchain** means the payment is permanently, publicly, and independently verifiable by anyone — including your hackathon judges, who can look up the transaction ID themselves.

Chapter 7 — Understanding Algorand

What is Algorand?

Algorand is a public blockchain — a shared, digital ledger maintained by thousands of computers around the world instead of one company — designed to be **fast, cheap, and final** (once a transaction is confirmed, it cannot be reversed or double-spent). Transactions typically confirm in a few seconds and cost a tiny fraction of a rupee, which makes it practical for the kind of **frequent, small, per-API-call payments** that x402 needs.

Why this hackathon uses Algorand

- **Speed** — confirmation in seconds, not minutes, so your Router isn't stuck waiting while a user watches a loading spinner.
- **Low, predictable fees** — paying an agent ₹2 for a hotel search only makes sense if the network fee is a tiny fraction of that ₹2.
- **Instant finality** — once a transaction is confirmed, it is done, permanently — no waiting for multiple confirmations like some other blockchains require.
- **Native stablecoin support (ASAs)** — Algorand has first-class support for tokens like USDC directly at the protocol level (explained below), so you don't need complex extra smart contracts just to move stable-value money.
- **Good developer SDKs** — official SDKs exist for JavaScript/TypeScript, Python, Go, and Java, so it's easy to plug into a Node.js or React-based hackathon stack.

Core terms, explained simply

Blockchain. A shared list of records ("blocks") that is copied across many computers and cryptographically linked together, so no one can secretly change an old record without everyone else noticing.

Transaction. A single recorded action on the blockchain — most commonly "send X amount of currency from wallet A to wallet B." Every transaction is signed by the sender's private key.

Wallet. A pair of cryptographic keys: a **public address** (like a bank account number — safe to share) and a **private key** (like a PIN — must be kept secret). Whoever holds the private key can authorize transactions from that wallet.

USDC ASA. ASA stands for **Algorand Standard Asset** — Algorand's built-in way to represent any token (not just Algorand's native currency, ALGO) directly on the blockchain. **USDC** is a stablecoin, a token designed to always be worth approximately \$1 USD. "USDC ASA" simply means "USDC represented as a standard Algorand token," which is what your agents will actually be paid in, since it's far more predictable than a volatile cryptocurrency.

Testnet. A separate, free-to-use copy of the Algorand network used for testing. Testnet ALGO and testnet USDC have **no real-world value** — you get them free from a "faucet" (a website that gives out small amounts for testing). **Use testnet for your entire hackathon build.**

Mainnet. The real, live Algorand network where transactions use real money with real value. You would only move to mainnet after your product is fully built, tested, and ready for real users — almost certainly out of scope for the hackathon itself.

Transaction ID (TxID). A unique fingerprint generated for every confirmed transaction. Anyone can take a Transaction ID and look it up on a public block explorer (like Algorand's testnet explorer) to see exactly what happened: who paid whom, how much, and when.

Why the Transaction ID proves payment

Because the blockchain is public and tamper-evident, a Transaction ID is **independently verifiable** — the server doesn't have to trust the client's word that "I paid," and your hackathon judges don't have to trust your word either. Anyone, at any time, can paste that Transaction ID into a block explorer and see the real, permanent record. This is fundamentally different from a screenshot of a bank transfer, which can be faked — a blockchain transaction cannot be faked without controlling the sender's private key.

A minimal Algorand payment in JavaScript (testnet, teaching example)

```
import algosdk from "algosdk";

// Connect to Algorand TestNet via a public node provider
const algod = new algosdk.Algodv2(
  "", // no token needed for many public testnet endpoints
  "https://testnet-api.algonode.cloud",
  ""
);

async function sendPayment(senderAccount, receiverAddress, amountMicroAlgos) {
  const params = await algod.getTransactionParams().do();

  const txn = algosdk.makePaymentTxnWithSuggestedParamsFromObject({
    from: senderAccount.addr,
    to: receiverAddress,
    amount: amountMicroAlgos, // 1 ALGO = 1,000,000 microAlgos
    suggestedParams: params,
  });

  const signedTxn = txn.signTxn(senderAccount.sk);
  const { txId } = await algod.sendRawTransaction(signedTxn).do();

  await algosdk.waitForConfirmation(algod, txId, 4);
  return txId; // <-- this is your proof of payment
}
```

For USDC (an ASA) instead of native ALGO, you would use

`makeAssetTransferTxnWithSuggestedParamsFromObject` with USDC's testnet asset ID — the shape of the flow is the same.

Chapter 8 — Understanding the Facilitator

What is the Facilitator?

The **Facilitator** is a small, trusted helper service that sits between the agent server and the Algorand blockchain. Its whole job is: **take a signed payment from a client, check it's valid, and settle it on-chain — so the agent server itself never has to know the details of blockchain signatures or transaction construction.**

Think of the Facilitator like a **card payment terminal at a shop**. The shopkeeper doesn't personally understand how Visa's international banking network verifies your card — they just swipe the card, and the terminal handles all the complex verification and comes back with "approved" or "declined." The Facilitator plays that exact role for x402 + Algorand payments.

Why it exists

Without a Facilitator, **every single agent server** would need to independently implement: blockchain signature verification, transaction construction, network calls to Algorand nodes, and error handling for a dozen edge cases. That's a huge amount of duplicated, security-sensitive code across every agent. The Facilitator centralizes that complexity into one well-tested service that every agent can call with a simple internal API — usually just `POST /verify` and `POST /settle`.

How it checks signatures

When the client signs a payment authorization, it produces a cryptographic signature mathematically tied to both (a) the exact payment details (amount, recipient, nonce) and (b) the sender's private key. The Facilitator uses standard Algorand SDK functions to check: **does this signature really correspond to this sender's public address, for exactly this payment data, with nothing altered?** If even one byte of the payment details changes after signing, the signature check fails — this is what makes tampering detectable.

How it settles payments

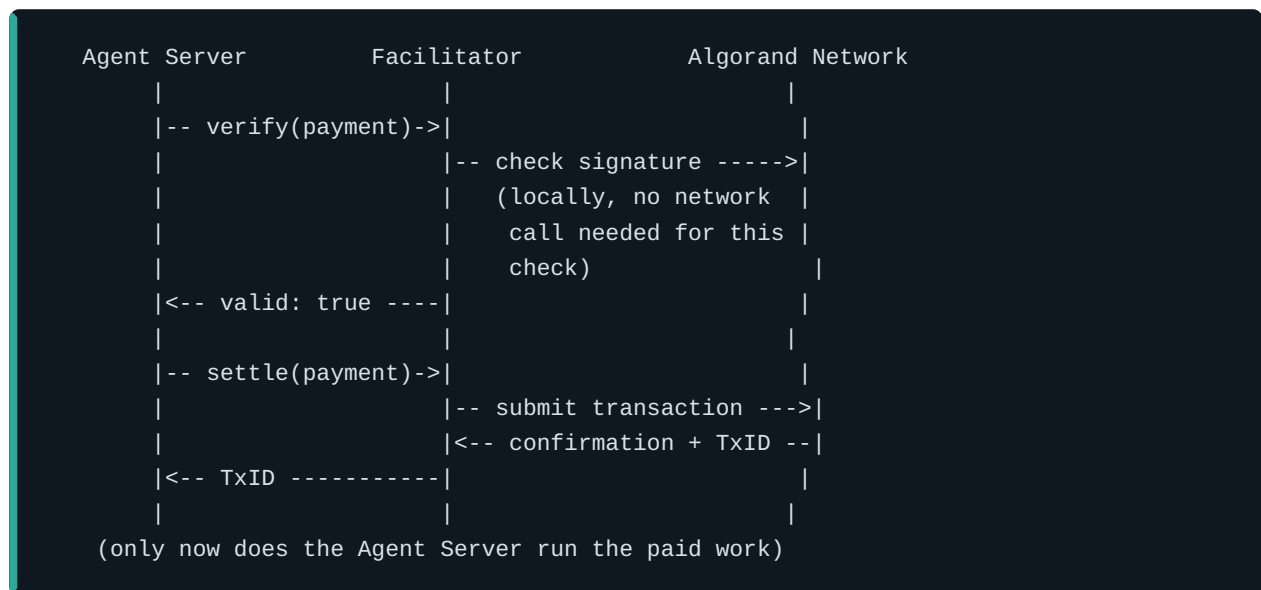
Once verification passes, the Facilitator builds the actual Algorand transaction from the payment details, submits it to an Algorand node, and waits for confirmation. It then returns the **Transaction ID** back to the agent server, which uses it as proof to include in the receipt.

How it prevents fake payments

1. **Signature verification** rejects anything not actually signed by the claimed sender.

2. **Nonce checking** rejects a payment proof that has been submitted before (stopping "replay" reuse of the same payment for multiple free calls).
3. **Amount/recipient matching** rejects a signed payment that doesn't exactly match what was actually challenged (a client can't sign for ₹1 and try to use it against a ₹10 challenge).
4. **On-chain settlement** means the Facilitator itself cannot lie about a payment happening — the blockchain's own consensus (thousands of independent nodes) is the final judge of whether the transaction is real.

Sequence diagram: verify + settle



Chapter 9 — Complete System Architecture

The full picture


```
debugging) |
```

```
+-----+
```

Every connection, explained

- **User** → **Frontend** → **Router**: the user types a request in a simple React web app; the frontend sends it as one HTTP POST to the Router's `/task` endpoint.
- **Router** → **Agents**: the Router calls each specialist agent's API directly over HTTP, following the x402 challenge-and-retry flow from Chapter 6.
- **Router** → **Payment Module**: whenever an agent responds `402`, the Router hands the challenge to its internal Payment Module, which holds (or accesses) the Router's Algorand wallet and produces a signed payment.
- **Payment Module** → **Facilitator**: the signed payment is sent to the Facilitator to be verified and settled on-chain.
- **Facilitator** → **Algorand Blockchain**: the Facilitator submits the real transaction and waits for confirmation, then hands back a Transaction ID.
- **Router/Agents** → **Database**: every task, every agent call, every payment, and every receipt is written to the Database so nothing is lost and everything can be looked up later.
- **Everything** → **Audit Logs**: a simpler, append-only log of every important event (request received, payment challenged, payment settled, response returned) — useful for debugging live during your demo, and great to show judges as proof of transparency.

Why this shape is a good hackathon architecture

It cleanly separates **four concerns** that are easy to reason about and easy to divide between two teammates: 1. **Orchestration** (Router) — who does what, when. 2. **Specialized work** (Agents) — the actual AI tasks. 3. **Trust and money** (Payment Module + Facilitator + Algorand) — proving and moving value. 4. **Memory** (Database + Audit Logs) — remembering everything that happened, for both the app and for your judges.

Chapter 10 — API Design

Design principles

Keep the Router's public API tiny — just four endpoints. Every complex thing (payment challenges, agent selection, blockchain calls) should happen **inside** the Router, hidden from whoever is calling it.

POST /task

Kick off a new multi-agent job.

Request

```
{
  "goal": "Plan a 3-day trip to Goa under 20000 rupees",
  "userId": "user_8231"
}
```

Response (202 Accepted — task started)

```
{
  "taskId": "task_9f31ab",
  "status": "IN_PROGRESS"
}
```

Headers: `Content-Type: application/json`, `Authorization: Bearer <user token>`

POST /pay

Used internally by the Router (or exposed for manual testing) to trigger the x402 payment flow for one specific agent call.

Request

```
{
  "taskId": "task_9f31ab",
  "agent": "hotel-agent",
  "challenge": {
    "amount": "2.00",
    "currency": "USDC",
    "payTo": "ALGO_ADDRESS_HERE",
    "nonce": "8f0a1c93"
  }
}
```

Response (200 OK)

```
{
  "txId": "ALGO_TXID_HERE",
  "status": "SETTLED"
}
```

GET /receipt/:taskId

Fetch a full, itemized proof of every payment made for a task.

Response (200 OK)

```
{
  "taskId": "task_9f31ab",
  "payments": [
    { "agent": "flight-agent", "amount": "5.00", "txId": "TXID_1" },
    { "agent": "hotel-agent", "amount": "2.00", "txId": "TXID_2" }
  ],
  "total": "7.00",
  "currency": "USDC"
}
```

GET /status/:taskId

Check the live progress of a task (useful for showing a progress bar in the frontend).

Response (200 OK)

```
{
  "taskId": "task_9f31ab",
  "status": "IN_PROGRESS",
  "completedAgents": ["flight-agent"],
  "pendingAgents": ["hotel-agent", "weather-agent"]
}
```

Error handling conventions

Use standard HTTP status codes consistently so any teammate (or judge inspecting your API) instantly understands what happened:

Status	Meaning in this project
200 OK	Request succeeded fully
202 Accepted	Task started, still processing
400 Bad Request	Malformed input from the client
402 Payment Required	x402 payment challenge (used by agents, not the Router's own endpoints)
404 Not Found	Unknown taskId
409 Conflict	Payment nonce already used (possible replay attempt)
502 Bad Gateway	A required downstream agent failed
503 Service Unavailable	Facilitator or Algorand node unreachable

Every error response should follow one consistent shape:

```
{ "error": true, "code": "AGENT_UNAVAILABLE", "message": "hotel-agent did not
respond in time" }
```

Chapter 11 — Business Model

Why this hackathon requires pay-per-use

The whole point of x402 is proving that AI agents can transact **without subscriptions**. A pay-per-use model is not just a business choice here — it's the natural consequence of the technology: since every single API call can carry its own tiny payment, there's no need to bundle usage into a monthly plan at all.

Subscription vs. Pay-per-use

	Subscription	Pay-per-use
When you pay	Fixed amount every month, regardless of usage	Only when you actually use the service
Predictability	Predictable cost, unpredictable value (you might not use it much)	Cost scales exactly with value received
Best for	Frequent, steady usage	Occasional, bursty, or unpredictable usage
Setup friction	Sign-up, card details, cancellation flow	None — payment is the authorization
Fits AI agents?	Poorly — an agent might use a tool once a year	Perfectly — pay exactly for what's used

Why companies like pay-per-use

- **No wasted spend** — they never pay for a tool sitting idle.
- **Lower barrier to try new tools** — testing a new agent costs pennies, not a monthly commitment.
- **Costs scale with revenue** — if a company's own usage grows, so does their cost, but proportionally to the value they're extracting, not before it.
- **No procurement overhead** — no vendor contracts to negotiate for a five-rupee API call.

Example pricing

Service	Price per call
Hotel search	₹2
Flight search	₹5
Insurance verification	₹10
Full trip planning (bundled)	₹15

Pricing this low is only *possible* because settlement is instant and fees are tiny on Algorand — try charging ₹2 through a traditional card network and the processing fee alone might exceed the price.

Chapter 12 — Real Paying Users

Who would actually pay for this?

Customer	Why they would pay
Businesses	Automate multi-step workflows (customer support, logistics) without hiring more staff or committing to SaaS contracts they may outgrow or underuse.
Developers	Want to plug a paid capability (translation, OCR, search) into their own app instantly, without negotiating a contract with each provider.
AI companies	Want their own agents to be able to autonomously "hire" other companies' specialized agents — this is the direct target market for x402 itself.
Travel companies	Need live, accurate, multi-source data (flights, hotels, weather) assembled per customer request — pay-per-search matches their own per-booking revenue model.
Hospitals	Need to call specialized, often expensive, verification tools (insurance checks, drug interaction checks) only when a patient actually needs them.
Law firms	Need document analysis or case-law lookup tools used in bursts around case deadlines, not steadily every day — pay-per-use avoids idle subscription cost.
Government	Needs auditable, provable transaction trails for public accountability — a blockchain-settled payment system provides that by default.

The common thread

Every one of these customers has **bursty, unpredictable, or accountability-sensitive** usage patterns — exactly the conditions where "pay only for what you use, and prove exactly what you paid for" beats a flat subscription.

Chapter 13 — Unique Innovation Ideas

Beyond the basic "route and pay" flow, these ideas make your project stand out to judges:

Smart routing — instead of always calling the same fixed agent for a job type, the Router evaluates multiple candidate agents and picks the best one for this specific request.

Cheapest agent selection — when multiple agents can do the same job (e.g. three different hotel-search providers), automatically pick the one currently charging the lowest price.

Fastest agent selection — track historical response times per agent and prefer the fastest one when the user needs a quick answer.

Agent reputation score — track each agent's success rate, average response time, and dispute rate over time, and use this score to influence routing decisions — agents that fail often get chosen less.

Automatic fallback — if a chosen agent fails or times out, automatically retry with a backup agent instead of failing the whole task.

Dynamic pricing — let agents adjust their own price based on demand (e.g. slightly higher price during peak load), similar to surge pricing.

Load balancing — spread requests for the same job type across multiple equivalent agent instances so no single agent gets overwhelmed.

Fraud detection — flag unusual payment patterns (e.g. the same nonce reused, abnormally large request bursts from one wallet) for review.

Agent marketplace — let any developer register a new agent with the Router (its price, its capability, its wallet address), turning your Router into an open marketplace rather than a fixed set of hard-coded agents.

Payment analytics — a dashboard showing total spend per task type, per agent, over time — useful both for your demo and as a real product feature for business customers.

Visual workflow builder — a drag-and-drop UI letting a non-technical user define their own multi-agent workflow (which agents run, in what order) without writing code.

You do not need to build all of these for the hackathon — pick two or three that you can demo convincingly, and mention the rest in your "Future Scope" slide (Chapter 23) as evidence of a bigger vision.

Chapter 14 — Technology Stack

Layer	Technology	Why
Frontend	React	Component-based, huge ecosystem, fast to build a demo UI with.
Backend / Router	Node.js + Hono	Hono is a lightweight, extremely fast web framework — perfect for a hackathon API layer with minimal boilerplate.
AI Agents	Node.js services (or Python, if a teammate is more comfortable there) calling an LLM API for the "intelligence" part of each agent	Keeps each agent as an independent, small, focused service.
Database	PostgreSQL (or SQLite for a fast local hackathon setup)	Reliable relational storage for tasks, payments, and receipts, with straightforward transaction support.
Blockchain	Algorand (Testnet)	Fast, cheap, final settlement — see Chapter 7.
Algorand SDK	<code>algosdk</code> (JavaScript/TypeScript)	Official SDK; handles transaction construction, signing, and submission.
x402 libraries	x402 reference middleware/client libraries (check the latest official x402 GitHub repo for your language)	Implements the challenge/retry/verify flow from Chapter 6 so you don't hand-roll HTTP header parsing yourself.
Facilitator	A small dedicated Node.js/Hono service (can be self-hosted for the hackathon, or a hosted x402 facilitator if available for Algorand)	Centralizes payment verification/settlement logic — see Chapter 8.

Why Hono specifically: it is much lighter than Express, has first-class TypeScript support, and runs anywhere (Node.js, Bun, Cloudflare Workers) — handy if you want to quickly deploy an agent or the Router to a serverless platform during the hackathon.

Chapter 15 — Folder Structure

```

atomic-router/
├── apps/
│   ├── frontend/                                # React app (user-facing UI)
│   │   ├── src/
│   │   │   ├── components/                    # Reusable UI pieces (TaskForm, ReceiptView...)
│   │   │   ├── pages/                        # Top-level screens
│   │   │   └── api/                          # Functions that call the Router's REST API
│   │   └── package.json
│   ├── router/                                # The Router service (Hono)
│   │   ├── src/
│   │   │   ├── routes/                      # /task, /pay, /receipt, /status handlers
│   │   │   ├── planner/                    # Logic that decides which agents are needed
│   │   │   ├── payment/                   # x402 client logic (challenge -> sign -> retry)
│   │   │   ├── db/                        # Database models & queries
│   │   │   └── index.ts
│   │   └── package.json
│   ├── facilitator/                          # Payment verification + settlement service
│   │   ├── src/
│   │   │   ├── verify.ts
│   │   │   ├── settle.ts
│   │   │   └── index.ts
│   │   └── package.json
│   └── agents/                               # One folder per specialist agent
│       ├── flight-agent/
│       ├── hotel-agent/
│       ├── weather-agent/
│       └── restaurant-agent/
│           ├── src/
│           │   ├── index.ts                # Exposes the agent's paid HTTP endpoint
│           │   ├── x402-guard.ts          # Middleware: reply 402 until payment proven
│           │   └── logic.ts               # The actual specialist AI/logic
│           └── package.json
├── packages/
│   ├── x402-shared/                          # Shared x402 types & helper functions
│   ├── algorand-wallet/                     # Shared Algorand wallet/signing helpers
│   └── types/                               # Shared TypeScript types across all apps
├── db/
│   └── migrations/                          # SQL migration files (see Chapter 16)
├── docs/
│   └── this-handbook.pdf
├── .env.example                             # Template for required secrets/config
├── docker-compose.yml                       # Optional: spin up DB + all services together
└── README.md

```

Why this shape works well for a two-person hackathon team: one teammate can own `apps/router` + `apps/facilitator` + `packages/algorand-wallet` (the payment/blockchain side), while the other owns `apps/agents/*` + `apps/frontend` (the AI/UX side) — with `packages/` as the shared contract between them, so both sides can be built in parallel without blocking each other.

Chapter 16 — Database Design

users

Column	Type	Notes
id	UUID (PK)	
name	text	
email	text	unique
wallet_address	text	the user's or org's Algorand address, if applicable
created_at	timestamp	

agents

Column	Type	Notes
id	UUID (PK)	
name	text	e.g. "hotel-agent"
endpoint_url	text	where the Router calls it
price_per_call	numeric	in USDC
wallet_address	text	Algorand address the agent gets paid to
reputation_score	numeric	see Chapter 13
status	text	ACTIVE , OFFLINE , DISABLED
created_at	timestamp	

tasks

Column	Type	Notes
id	UUID (PK)	
user_id	UUID (FK -> users.id)	
goal	text	the raw user request
status	text	IN_PROGRESS , SUCCESS , FAILED , ROLLED_BACK
created_at	timestamp	
completed_at	timestamp	nullable

transactions

Column	Type	Notes
id	UUID (PK)	
task_id	UUID (FK -> tasks.id)	
agent_id	UUID (FK -> agents.id)	
status	text	PENDING , PAID , FAILED , REFUNDED
created_at	timestamp	

payments

Column	Type	Notes
id	UUID (PK)	
transaction_id	UUID (FK -> transactions.id)	
amount	numeric	
currency	text	e.g. USDC
nonce	text	unique — prevents replay (see Chapter 18)
algorand_tx_id	text	on-chain proof, nullable until settled
settled_at	timestamp	nullable

receipts

Column	Type	Notes
id	UUID (PK)	
task_id	UUID (FK -> tasks.id)	
total_amount	numeric	
currency	text	
generated_at	timestamp	

logs

Column	Type	Notes
id	UUID (PK)	
task_id	UUID (FK -> tasks.id, nullable)	
level	text	INFO , WARN , ERROR
message	text	
metadata	jsonb	free-form extra detail
created_at	timestamp	

Design notes: - `nonce` is marked **unique** at the database level — this is your simplest, most reliable defense against replayed payments (Chapter 18). - `transactions` sits between `tasks` and `payments` so that one task's atomicity (all-or-nothing across several agent calls) can be checked with one query: "are all transactions for this task_id in status PAID?" - Keep `logs` append-only (never update or delete rows) — this is what makes your audit trail trustworthy.

Chapter 17 — Error Handling

Failure	What it means	How to handle it
Payment failed	The Facilitator rejected the signature, or the wallet had insufficient balance.	Do not retry the agent call. Mark the transaction FAILED , log it, and either try a backup agent or fail the whole task (rollback).
Agent offline	The agent's endpoint didn't respond (connection refused / timeout).	Mark agent OFFLINE in the agents table, route to a backup agent if one exists, otherwise fail the task. Never charge for an agent that never responded.
Wrong signature	The signed payment doesn't match the challenge, or was signed by the wrong key.	Reject immediately with 409 Conflict or 400 Bad Request . This may indicate a bug in your client's signing code — log full details for debugging.
Blockchain timeout	The Algorand transaction was submitted but confirmation is taking unusually long.	Poll for confirmation with a sensible timeout (e.g. 10 seconds); if it doesn't confirm in time, treat as PENDING , notify the user the task is still processing, and reconcile automatically once (if) it confirms.
Network issue	A general connectivity failure between any two services (Router ↔ Agent, Router ↔ Facilitator, Facilitator ↔ Algorand node).	Use retries with exponential backoff for transient failures (2–3 attempts), then fail gracefully with a clear error message rather than hanging indefinitely.
Rollback needed	One or more required agents failed after others already succeeded and were paid.	Mark the task ROLLED_BACK . Since Algorand payments cannot be reversed once settled, "rollback" here means: refund the affected agent payment as a new, separate transaction back to the user's wallet, and clearly record both the original payment and the refund in the payments table for a full audit trail.

General principle: always fail loudly and specifically (a clear error code + message written to **logs**), never silently. In an autonomous, unattended multi-agent system, a silent failure is far more dangerous than a loud one — no human is watching every request to notice quietly wrong behavior.

Chapter 18 — Security

Signature verification. Every payment must be checked cryptographically against the sender's real public address before being trusted — never trust a client's self-reported "I paid" claim (see Chapter 8).

Replay attacks. A malicious client could try to reuse a previously-signed, previously-accepted payment proof to get a second free response. Defend against this with a **unique nonce per challenge**, enforced as a unique database constraint, so any reused nonce is instantly rejected.

Payment verification. Never do the paid work before the Facilitator confirms the payment is genuinely valid and settling — see the ordering in Chapter 6's flow diagram; skipping ahead defeats the entire point of x402.

API authentication. The Router's own public endpoints (`/task` , `/status` , `/receipt`) should still require normal API authentication (e.g. a bearer token per user) — x402 secures the *agent-to-agent payment* layer, not the *user-to-Router* layer, which needs its own standard auth.

Wallet security. Never hard-code private keys in source code or commit them to git. Store them in environment variables or a secrets manager, and for a hackathon demo, use a **dedicated testnet-only wallet** holding only small testnet amounts — never reuse a wallet that also holds real funds.

Rate limiting. Apply per-client rate limits on the Router's public endpoints to prevent a single misbehaving or malicious caller from overwhelming your system or draining your demo wallet by triggering excessive real (even if testnet) payments.

Encryption. Use HTTPS/TLS for every service-to-service connection (Router ↔ Agent, Router ↔ Facilitator), and encrypt sensitive database columns (like `wallet_address` , if it maps to a private key reference) at rest.

Chapter 19 — Demo Flow (5 Minutes, Minute-by-Minute)

Minute 0:00 – 0:45 — The user request. Open the React frontend live. Type a real request, e.g. *"Plan a 3-day trip to Goa under ₹20,000."* Say out loud: *"This single request is about to trigger multiple independent AI agents, each paid automatically, in real time, on a public blockchain."*

Minute 0:45 – 1:30 — The payment challenge. Switch to a terminal/logs view showing the Router's console. Point out the moment an agent responds `402 Payment Required` and explain in one sentence: *"This agent won't do any work until it's paid — no accounts, no API keys, just this one HTTP exchange."*

Minute 1:30 – 2:15 — Signing and retry. Show the payment being signed (log line or a small visual) and the exact same request being retried with the `X-PAYMENT` header attached. Emphasize: *"This is machine-to-machine payment happening with zero human clicks."*

Minute 2:15 – 3:00 — The blockchain transaction. Open the Algorand **testnet block explorer** in a browser tab, paste in the live Transaction ID your system just produced, and show the confirmed transaction on-screen. This is your most powerful "wow" moment — a judge can see, independently, that real settlement happened.

Minute 3:00 – 3:45 — Agents working in parallel. Show your logs or a simple dashboard view of multiple agents (Flight, Hotel, Weather) responding, each with its own payment already confirmed. Point out that these ran **concurrently**, not one after another.

Minute 3:45 – 4:15 — The final response. Switch back to the frontend and show the combined, assembled itinerary appearing for the user — clean, complete, and ready to read.

Minute 4:15 – 4:45 — The receipt. Click "View Receipt" and show the itemized breakdown: which agent, how much, and the Transaction ID for each — proving exactly what was paid for what.

Minute 4:45 – 5:00 — The atomic guarantee (bonus, if time allows). If you have time, deliberately trigger a failure (e.g. kill one agent process) on a second, live run, and show the system correctly rolling back / failing gracefully rather than showing a broken half-answer. This single moment can be the difference between a good demo and a winning one.

Tip: rehearse this exact sequence at least three times before presenting. Have a **backup recorded video** of a successful run in case live testnet/network conditions misbehave during judging.

Chapter 20 — Presentation Slides (8 Slides)

Slide 1 — Problem

Say: "Today, AI agents cannot pay each other. Every paid API requires a human to sign up, enter a card, and manage a subscription — which makes truly autonomous, multi-agent AI systems impossible to build at scale." **Show:** one simple visual — a human blocking the path between two robot icons.

Slide 2 — Current Issues

Say: "Existing payment systems like Stripe and PayPal are built for humans, not machines. API keys require pre-existing relationships. None of this works for an AI agent that wants to try a brand-new tool for a single call, right now." **Show:** a short comparison list (API keys / subscriptions / manual checkout) each with a red X.

Slide 3 — Solution

Say: "We built an Atomic Multi-Agent Service Router: one request in, multiple specialist AI agents coordinated automatically, each paid instantly and provably via x402 on Algorand — and the whole operation is atomic, so nothing is ever half-paid or half-done." **Show:** your architecture diagram, simplified to 5 boxes.

Slide 4 — Architecture

Say: walk through the diagram from Chapter 9 in under 60 seconds: Router → Agents → Payment Module → Facilitator → Algorand → Database. **Show:** the full architecture diagram.

Slide 5 — Payment Flow

Say: walk through the x402 challenge → sign → retry → verify → settle → respond flow from Chapter 6, in plain language, in under 45 seconds. **Show:** the 7-step flow diagram.

Slide 6 — Demo

Say: "Let's see it live." (Switch to your live demo per Chapter 19, or play your backup video.) **Show:** live product / video.

Slide 7 — Business Model

Say: "This isn't just a hackathon toy — pay-per-use fits how AI agents will actually consume services: bursty, unpredictable, and without the overhead of subscriptions. [Give one or two customer examples from Chapter 12.]" **Show:** the pricing table from Chapter 11.

Slide 8 — Future Scope

Say: "This architecture scales beyond travel — into an open agent marketplace, enterprise automation, healthcare, and eventually a broader autonomous AI economy." (Chapter 23.) **Show:** a short 4–5 item roadmap list.

Delivery tip: keep total speaking time under 4 minutes for these 8 slides so you have buffer time for questions — judges remember teams that respect the clock.

Chapter 21 — Judges' Questions (50+) with Model Answers

Group A questions are about **core concepts**; Group B about **technical implementation**; Group C about **business & trust**; Group D are **hard/skeptical** questions you should specifically rehearse.

Group A — Core Concepts

1. Why Algorand? Because it offers fast (few-second), low-fee, final settlement, which is essential when payments are small (a few rupees) and frequent — a slower or more expensive chain would make micro-payments impractical.

2. Why x402 instead of building your own payment API? x402 is an open, emerging standard — building a proprietary payment scheme would mean every agent we want to interoperate with in the future would need custom integration work. Using x402 means any x402-compatible agent, from any team, could plug into our Router immediately.

3. Why "atomic"? Why does it matter here specifically? Because money is involved. If one agent in a multi-agent chain fails after others already succeeded, an atomic guarantee ensures the user is never charged for value they didn't receive, and the system's state never gets stuck half-completed.

4. Why not just use ChatGPT / one big AI model for everything? A single generalist model has no clean way to independently verify each specialized sub-task, cannot price different types of work differently, and has no fault isolation — if it gets one part wrong, everything is affected. Specialist agents each stay narrowly focused, which improves accuracy and lets each be priced, scaled, and improved independently.

5. Why not Stripe or PayPal for the payments? Both require pre-existing human accounts and card details; neither is designed for a machine to pay another anonymous machine for a single API call in one HTTP round-trip with no prior relationship — see Chapter 5 for the full comparison.

6. How is payment verified? The client signs a payment matching the server's challenge; the Facilitator independently checks the cryptographic signature and settles the transaction on Algorand before the server does any paid work — see Chapters 6 and 8.

7. What happens if an agent fails mid-task? The task is not marked successful; any payments made for that logical transaction are either not finalized or refunded, and the Router either retries with a backup agent or fails the whole task cleanly, per our atomic guarantee (Chapter 2).

8. How is rollback implemented? Since on-chain settlement can't be reversed once confirmed, rollback means issuing a new, separate refund transaction back to the payer, and recording both the original payment and refund transparently in our **payments** table.

9. How do you ensure trust between agents that have never interacted before? Trust doesn't come from a prior relationship — it comes from the payment itself being cryptographically provable and settled on a public blockchain that anyone (including a competing agent) can independently verify.

10. What is the role of the Facilitator, exactly? It centralizes the security-sensitive logic of verifying signatures and settling on-chain transactions, so individual agent servers don't each need to reimplement blockchain logic — see Chapter 8.

Group B — Technical Implementation

11. What blockchain network are you using — testnet or mainnet? Testnet, for the entire hackathon build — safe, free, and functionally identical to mainnet for demonstration purposes.

12. What currency do agents get paid in? USDC, represented as an Algorand Standard Asset (ASA) — a stablecoin, so pricing stays predictable in rupee/dollar terms rather than fluctuating with a volatile cryptocurrency.

13. How do you prevent replay attacks? Every payment challenge includes a unique, single-use nonce, enforced as a unique constraint at the database level — a reused nonce is rejected immediately.

14. What happens if two payments arrive with the same nonce at the same time? The database's unique constraint on `nonce` guarantees only the first insert succeeds; the second is rejected as a conflict (`409`).

15. How does the Router decide which agents to call? A planning step parses the user's goal and matches it against known agent capabilities (Chapter 4); this can range from simple keyword/intent matching up to LLM-based planning, depending on how far you take it.

16. What if two agents can do the same job? The Router can apply smart-routing logic — picking the cheapest, fastest, or highest-reputation agent for that call (Chapter 13).

17. How do agents know how much to charge? Each agent's price is configured (and can be dynamically adjusted) and returned in its `402` challenge response — the Facilitator and Router never need to know prices in advance.

18. What database are you using, and why? PostgreSQL (or SQLite for local hackathon speed) — a relational database gives us straightforward transactional guarantees and easy relationships between tasks, transactions, and payments (Chapter 16).

19. How do you handle a slow/stuck blockchain confirmation? We poll for confirmation with a sensible timeout, mark the transaction `PENDING` if it doesn't confirm quickly, and reconcile automatically once (if) it eventually confirms (Chapter 17).

20. What SDK are you using to talk to Algorand? The official `algosdk` JavaScript/TypeScript SDK, for building, signing, and submitting transactions.

21. Is your Router synchronous or does it call agents in parallel? Independent agents (ones that don't depend on each other's output) are called in parallel to keep total response time low; dependent agents are sequenced appropriately.

22. How do you test this without spending real money? Everything runs on Algorand Testnet using free faucet funds — functionally identical behavior to mainnet, with zero real financial risk.

23. What happens if the Facilitator itself goes down? Agent servers would return `503 Service Unavailable` for new paid requests; in a production system you'd run multiple Facilitator instances behind a load balancer for redundancy — noted as a future-scope hardening step.

24. How do you store private keys securely? Never in source control — via environment variables or a secrets manager, and for this hackathon, a dedicated testnet-only wallet holding only small test amounts (Chapter 18).

25. What happens if a malicious actor tries to fake a payment? Their signature check fails at the Facilitator before any paid work happens — a payment cannot be "faked," only genuinely signed by the true key-holder or rejected.

Group C — Business & Trust

26. Who is your actual paying customer? Businesses, developers, AI companies, and specific verticals like travel, healthcare, and legal — anyone with bursty or unpredictable usage of specialized paid tools (Chapter 12).

27. Why would a business prefer pay-per-use over a subscription? No wasted spend on unused capacity, lower barrier to trying new tools, and costs that scale proportionally with actual value received (Chapter 11).

28. How do you make money as the platform? Options include a small router/facilitation fee per transaction, or a percentage margin built into bundled multi-agent workflows — worth stating clearly which one your team is proposing.

29. Isn't blockchain overkill for a ₹2 payment? No — that's exactly the use case blockchains like Algorand were optimized for: settlement so cheap and fast that even sub-rupee payments remain economically sensible, which traditional card networks cannot achieve.

30. What stops competitors from copying this? The underlying protocol (x402) is intentionally open — our advantage is the quality of our routing intelligence, agent reputation system, and the ecosystem of agents we build around it, not a locked-down proprietary payment scheme.

31. Is this legal? Are there regulatory concerns with automated crypto payments? Using a stablecoin on testnet for a hackathon demo carries no regulatory exposure; a real production deployment on mainnet would need standard compliance review (KYC/AML considerations) — worth acknowledging honestly rather than dismissing.

32. How would you handle disputes (e.g. an agent takes payment but returns bad data)? This is where the agent reputation score (Chapter 13) matters — repeated poor-quality responses lower an agent's score and routing priority over time; for a stronger guarantee, a future version could add an escrow-style settlement that only releases payment after basic response validation.

33. What's your total addressable market? Any industry that already uses multiple specialized paid APIs/tools in a workflow — travel, insurance, healthcare, legal, logistics, customer support — a large and growing space as AI agents become more common.

Group D — Hard / Skeptical Questions

34. Isn't this just an API gateway with a payment gimmick bolted on? The payment layer isn't a gimmick — it's what makes zero-setup, per-call, machine-initiated transactions possible at all, which a traditional API gateway (built around long-lived API keys) fundamentally cannot do.

35. What if none of this actually needs blockchain — couldn't a central ledger do the same thing? A central ledger requires everyone to trust the operator of that ledger; a public blockchain lets any independent party (including a competing agent, or a hackathon judge) verify a payment happened without trusting us at all.

36. Why should I trust your Router isn't just pocketing the payment instead of paying the agent? Because every payment settles as its own independent, publicly verifiable Algorand transaction directly to the agent's wallet address — the Router itself never custodies user funds mid-transaction beyond authorizing the signed payment.

37. What happens at scale — thousands of agents, thousands of requests per second? The architecture already separates concerns (Router, Agents, Facilitator, DB) so each layer can scale horizontally independently; the main scaling consideration would be running multiple Facilitator/Algorand-node-access instances behind a load balancer.

38. How is this different from just calling multiple APIs with API keys in a loop? The paid-per-call model removes the need for any pre-existing API key relationship at all — any agent, from any provider, anywhere, can be called and paid instantly without prior integration setup.

39. What if a user disputes a charge after the fact — can they get their money back? Refunds are implemented as new, separate on-chain transactions back to the payer, fully logged — the original payment stays as an immutable historical record, while a refund provides the practical remedy.

40. Isn't atomicity across independent services actually impossible to fully guarantee (the "distributed transaction" problem)? Full ACID-style atomicity across independent networked services is genuinely hard — we approximate it with a practical compensating-transaction pattern (pay only after confirming success; refund/rollback on failure) rather than claiming a theoretically perfect distributed transaction, which is an honest and standard real-world approach.

Group E — Miscellaneous

41. Why Hono instead of Express? Lighter weight, faster, first-class TypeScript support, and easy to deploy in multiple environments — a good fit for quickly building several small services (Router, Facilitator, each Agent) during a hackathon.

42. Could this work with a different blockchain, like Ethereum or Solana? Yes — the x402 protocol itself is chain-agnostic; we chose Algorand specifically for its speed, low fees, and native ASA support, but the same architecture could target another chain with a different Payment Module implementation.

43. What's the average latency of a full task (request to final answer)? Depends on agent count and blockchain confirmation time, but state your own measured number from testing before demo day — judges respect specific measured numbers over vague claims.

44. How many agents do you currently have built? Answer honestly with your actual count, and mention the "agent marketplace" future-scope idea (Chapter 13) as the path to more.

45. What was the hardest part to build? Answer honestly — commonly it's getting the full x402 challenge/sign/retry/verify/settle loop working end-to-end correctly the first time.

46. Did you consider using an escrow model instead of pay-then-serve? Yes — it's a reasonable alternative worth exploring for further trust-hardening, mentioned in Group C, Q32, as future scope.

47. How do you know an agent's declared price is fair? Currently prices are set by each agent's owner; a future marketplace version could show comparative pricing and reputation to let the Router (and users) judge fairness themselves.

48. What happens if the user's wallet has insufficient funds? The Payment Module detects this before submission and the Router returns a clear `402`-adjacent error to the user/frontend rather than attempting and failing a blockchain transaction.

49. Is any part of this AI-generated data used to make real financial decisions? Be precise here: agents may use AI to interpret requests or produce recommendations, but the payment logic itself is deterministic code, not AI-generated — this separation is an important trust point to state clearly.

50. What would you build next if you had another week? Pick honestly from Chapter 23's future scope list, and be specific rather than vague — judges remember concrete next steps.

51. How is this audit-friendly for a business or regulator? Every request, payment challenge, signature, settlement, and receipt is logged in an append-only audit log and independently verifiable on a public blockchain — a stronger transparency guarantee than most traditional payment systems.

52. What's your biggest technical risk if this had to run in production tomorrow? Be honest — likely candidates are Facilitator single-point-of-failure, and the need for a more robust escrow/dispute-resolution mechanism — naming real risks builds more credibility than pretending there are none.

Preparation tip: don't try to memorize each answer word-for-word. Understand the *reasoning* behind each one (all rooted in Chapters 1–18) so you can answer confidently even if a judge asks a slightly different phrasing of these questions.

Chapter 22 — Comparison with Existing Tools

Tool	What it's good at	What it lacks (vs. our project)
ChatGPT (as a single assistant)	General conversation, broad reasoning	No native multi-agent specialization, no built-in payment layer for hiring other tools/agents automatically.
CrewAI	Orchestrating multiple AI agents with defined roles	Strong at agent orchestration, but has no native machine-to-machine payment protocol built in.
LangGraph	Building controllable, stateful multi-step agent workflows	Excellent for workflow logic/state machines, but again, no native payment settlement layer.
AutoGen	Multi-agent conversation frameworks	Focused on agents talking to each other, not on agents <i>paying</i> each other for real, provable, per-call work.
Zapier	Connecting apps/APIs via pre-built integrations	Requires pre-existing API key relationships and subscriptions with each connected app — the opposite of a zero-setup, pay-per-call model.
Stripe	Human-initiated payments, subscriptions, checkout flows	Not designed for autonomous, per-call, machine-initiated micro-payments without a pre-existing merchant/account relationship.
Traditional REST APIs (API-key based)	Simple, well-understood integration pattern	Requires sign-up and pre-arranged billing before first use — impossible for an agent to use a brand-new tool instantly and autonomously.

The core differentiator, in one sentence

Every one of these tools solves either "**how do multiple agents coordinate**" or "**how does a human pay for a service**" — our project is one of the few that solves **both at once**, specifically for autonomous, unattended, machine-to-machine usage, with an atomic guarantee that protects the payer even when parts of the workflow fail.

Chapter 23 — Future Scope

AI Service Marketplace. Open the Router up so any developer can register their own agent (with its own price and capability), turning the project from a fixed travel-planning demo into a general-purpose marketplace where any AI service can be discovered and paid for automatically.

Enterprise AI Platform. Package the Router + Facilitator + agent-marketplace as an internal platform large companies can deploy to let their own internal tools and external vendor tools transact automatically, with full audit trails for compliance.

Healthcare AI. Specialized agents for symptom triage, insurance verification, pharmacy stock checks, and appointment scheduling — all billed per-use, which fits how hospitals actually consume specialized tools (Chapter 12).

Government AI. Public-sector workflows (permit processing, document verification) benefit enormously from the built-in, publicly-auditable transaction trail that blockchain settlement naturally provides.

Financial AI. Agents for fraud checks, credit scoring lookups, or KYC verification, each independently priced and independently verifiable — a natural fit for an industry that already demands strong auditability.

Autonomous AI Economy. The long-term vision: a world where AI agents routinely discover, hire, and pay other AI agents to accomplish complex goals with no human in the loop for routine transactions — our Router is a working, concrete step toward that future, not just a concept.

Chapter 24 — Complete Development Roadmap

A realistic day-by-day (or session-by-session) plan for a hackathon team.

Step 1 — Set up the project

Initialize the monorepo structure from Chapter 15. Set up shared `packages/types` and `packages/algorand-wallet`. Create Algorand testnet wallets for the Router and each planned agent, and fund them via the testnet faucet.

Step 2 — Create the Router skeleton

Build the four core endpoints from Chapter 10 (`/task` , `/pay` , `/receipt` , `/status`) with mock (non-paid, hard-coded) agent responses first, just to prove the orchestration shape works end-to-end.

Step 3 — Integrate x402

Add the challenge/sign/retry logic to the Router's Payment Module, and add `402` -challenge middleware to one agent first (e.g. the Hotel Agent) as a proof of concept before rolling it out to the rest.

Step 4 — Integrate Algorand

Build the Facilitator's `verify` and `settle` endpoints, wire it to a real Algorand testnet node, and confirm you can see a real transaction appear on the public testnet block explorer.

Step 5 — Build the frontend

A simple React form for entering a request, a live status view, and a receipt view — keep the UI simple and legible; judges reward clarity over visual flourish in a hackathon demo.

Step 6 — Test the full payment flow end-to-end

Run the complete flow for all planned agents together — request → challenges → payments → settlement → aggregated response → receipt. Deliberately break one agent to confirm your atomic rollback behavior works as designed (Chapter 2, Chapter 17).

Step 7 — Deploy

Get the Router, Facilitator, and each Agent running on publicly reachable URLs (any simple hosting works for a hackathon — the point is that judges/testers can hit real endpoints, not just `localhost`).

Step 8 — Prepare the demo

Rehearse the 5-minute demo flow from Chapter 19 at least three times, prepare your 8 slides (Chapter 20), record a backup video of a successful run, and review the Judges' Questions in Chapter 21 as a team the night before.

Suggested time allocation for a typical ~24–36 hour hackathon

Phase	Approx. share of total time
Steps 1–2 (setup + router skeleton)	15%
Steps 3–4 (x402 + Algorand integration)	35%
Step 5 (frontend)	15%
Step 6 (end-to-end testing, including the rollback case)	15%
Steps 7–8 (deploy + demo prep)	20%

Leave real buffer time before your presentation slot — integration bugs between the payment layer and the blockchain layer are the most common place hackathon teams lose time, so budget generously for Steps 3–4 and don't skip Step 6's deliberate failure test.

You're ready to build.

Every concept a judge might ask about — atomicity, x402, Algorand, the Facilitator, the architecture, the business model — is now in one place. Keep this book open, split the work along the Router/Payment side and the Agents/Frontend side, and test the failure case (Chapter 17) as early as you test the success case.

Good luck — go build something that actually pays for itself.